

GCC1734 - INTELIGÊNCIA ARTIFICIAL

Trabalho 3

Prof. Eduardo Bezerra (ebezerra@cefet-rj.br)

CEFET/RJ

Escola de Informática e Computação
Bacharelado em Ciência da Computação

Junho/2025

Sumário

1	Predição de preços de diamantes (2 pts)	4
2	Redes Neurais MLP com PyTorch (2 pts)	4
2.1	Parte A – Classificação	5
2.2	Parte B – Regressão	5
3	Diagnóstico de qualidade (2 pts)	6
4	Conjuntos desbalanceados	7
4.1	Classificação (2 pts)	7
4.2	Regressão (2 pts)	7

Especificação da entrega

1. Você deve produzir um único *notebook* Jupyter. Esse *notebook* deve apresentar as **implementações e os resultados de execução** de cada parte desse trabalho. Nesse *notebook*, descreva **em detalhes** de que forma implementou cada parte desse trabalho.
2. Já é fornecido junto com este enunciado um *notebook* Jupyter para você usar como ponto de partida. Você encontra esse arquivo nesse [link](#).
3. Utilize a linguagem de marcação (*markdown*) para organizar seu *notebook* em seções que reflitam cada parte deste trabalho. Sua solução para cada parte deve estar facilmente encontrável nesse *notebook*.
4. O único arquivo a ser submetido é o *notebook* Jupyter. Esse arquivo deve ser nomeado com o seguinte padrão: GCC1734_T3_SEU_NOME_COMPLETO.ipynb. Um exemplo: GCC1734_T3_EDUARDO_BEZERRA_DA_SILVA.ipynb. Siga à risca essa convenção de nomenclatura.
5. Você deve também elaborar um vídeo (cuja duração aproximada foi especificada no primeiro dia de aula) no qual você deve explicar os aspectos mais importantes de cada parte do seu trabalho. Nesse vídeo, você também deve demonstrar a execução de cada parte e apresentar uma análise dos resultados obtidos. O link para acesso a esse vídeo deve estar contido na primeira célula (de texto) do *notebook* Jupyter.
6. Cuide para que a qualidade áudio-visual do seu vídeo esteja adequada para que seu conteúdo possa ser avaliado.
7. A entrega aqui especificada deve ser realizada pela plataforma MS Teams, até a data estabelecida. Trabalhos entregues com atraso irão sofrer desconto na nota (20% a cada dia de atraso).
8. Esse trabalho é individual. Você é livre para discutir com seus colegas de turma sobre as partes desse trabalho. Entretanto, você deve manter para si as soluções que produzir. Eventuais cópias em quaisquer partes do trabalho serão penalizadas com nota zero.

1 Predição de preços de diamantes (2 pts)

Nessa parte, você deve treinar um modelo de regressão sobre o conjunto de dados **Diamond**. Em particular, você deve criar um modelo para prever o valor do preço (representado na variável dependente **price**) de um diamante usando os demais atributos como variáveis independentes.

Repare que o conjunto de dados **Diamond** contém tanto atributos contínuos quanto categóricos. Sendo assim, você precisará realizar passos de pré-processamento sobre o conjunto de dados antes de iniciar o treinamento dos modelos. Para isso, reuse o código fornecido neste **notebook**. Com base nesse código, aplique os seguintes passos de pré-processamento:

1. Divida os dados em três conjuntos: treino (60%), validação (20%) e teste (20%);
2. Codifique os atributos categóricos;
3. Normalize os atributos contínuos.

Não existe um algoritmo de aprendizado de máquina que seja o melhor em todas as situações. Dessa forma, o que se faz na prática é testar diversos algoritmos candidatos e escolher o que gerar o melhor modelo, de acordo com alguma métrica de avaliação pré-estabelecida. Sendo assim, você irá criar modelos de predição de preços por meio dos seguintes algoritmos implementados na biblioteca Scikit-Learn:¹

1. `sklearn.linear_model.LinearRegression`
2. `sklearn.neighbors.KNeighborsRegressor`
3. `sklearn.ensemble.GradientBoostingRegressor`

Certifique-se de treinar todos os modelos sobre o mesmo conjunto de treinamento. Após o treinamento, você deve escolher quais dos três modelos gerados é o melhor. Para isso, utilize os dados do conjunto de validação. Utilize a métrica RMSE para realizar essa seleção.

Após escolher o modelo vencedor, você deve avaliar a qualidade preditiva desse modelo. Para isso, utilize os exemplos do conjunto de teste. Isso permitirá que você obtenha uma estimativa do quão efetivo (do ponto de vista preditivo) é o modelo vencedor.

1. Reporte o poder preditivo do modelo sobre o conjunto de teste. Como métricas de avaliação, use o RMSE e o coeficiente de determinação R^2 .
2. Apresente uma análise dos resultados obtidos. Como parte dessa análise, para o modelo vencedor, construa gráficos conforme os exemplificados neste [link](#).

2 Redes Neurais MLP com PyTorch (2 pts)

O objetivo desta parte é avaliar sua capacidade de projetar, treinar e interpretar modelos MLP (Perceptron Multicamadas) para problemas supervisionados de classificação e regressão.

¹Por simplicidade, você pode manter os valores *default* dos hiperparâmetros de cada algoritmo. Note, entretanto, que, em uma situação prática, os melhores valores para esses hiperparâmetros deveriam ser identificados também.

2.1 Parte A – Classificação

Implemente um modelo MLP em PyTorch para classificar amostras do conjunto **Wine** (disponível em `sklearn.datasets.load_wine`). Siga as etapas abaixo:

1. Divida os dados em três conjuntos: treino (60%), validação (20%) e teste (20%).
2. Normalize os atributos de entrada.
3. Crie um modelo MLP com pelo menos:
 - uma camada oculta com ativação **ReLU**;
 - função de perda **CrossEntropyLoss**;
 - otimizador **Adam**.
4. Treine o modelo por no mínimo 50 épocas e registre a acurácia no conjunto de teste.
5. Plote a curva de perda ao longo do treinamento.

No vídeo explique seu código-fonte, incluindo a avaliação final no conjunto de teste. Além disso, apresente a análise a curva de perda durante o treinamento.

2.2 Parte B – Regressão

Utilize o conjunto **California Housing** (acessível via `sklearn.datasets.fetch_california_housing`) para gerar um modelo de previsão do valor médio das casas.

1. Divida os dados em três conjuntos: treino (60%), validação (20%) e teste (20%).
2. Normalize os atributos de entrada.
3. Construa um MLP para regressão com pelo menos:
 - duas camadas ocultas com ativação **ReLU**;
 - função de perda **MSELoss**;
 - otimizador **Adam**.
4. Treine o modelo por no mínimo 100 épocas.
5. Calcule o erro quadrático médio (RMSE) no conjunto de teste.

No vídeo explique seu código-fonte e o resultado final do RMSE. Além disso, apresente a análise a curva de perda durante o treinamento.

3 Diagnóstico de qualidade (2 pts)

O artigo *Dataset Cartography: Mapping and Diagnosing Datasets with Training Dynamics* propõe o uso de *training dynamics* para visualizar e diagnosticar a qualidade dos exemplos em um conjunto de dados supervisionado. A partir da evolução da confiança de um modelo nas predições ao longo do treinamento, é possível classificar os exemplos em três grupos: *easy-to-learn*, *ambiguous* e *hard-to-learn*. Essa visualização, chamada de *data map*, utiliza duas métricas principais:

- **Confiança:** média da probabilidade atribuída à classe correta ao longo das épocas;
- **Variabilidade:** desvio padrão dessa probabilidade ao longo das épocas.
- **Corretude** (*correctness*): fração de épocas em que o modelo classificou corretamente a instância durante o treinamento.

Nesta parte, o objetivo é você construir um *data map* para o conjunto de dados **Wine** (disponível em `sklearn.datasets.load_wine`). Você deverá:

1. Divida os dados em três conjuntos: treino (60%), validação (20%) e teste (20%).
2. Treinar um modelo de classificação (usando uma rede neural MLP ou regressão logística) no conjunto de dados por múltiplas épocas;
3. Para cada instância de treino, registrar a probabilidade atribuída à classe correta ao final de cada época;
4. Calcular para cada instância:
 - A **confiança** (média das probabilidades da classe correta);
 - A **variabilidade** (desvio padrão dessas probabilidades).
 - A **corretude** (*correctness*).
5. Plotar um gráfico de dispersão (*data map*), com:
 - Eixo X: variabilidade;
 - Eixo Y: confiança;
 - Cores ou marcadores diferenciando os exemplos com diferentes faixas de corretude.
6. Plotar um gráfico de dispersão (*data map*), com:
 - Eixo X: variabilidade;
 - Eixo Y: confiança;
 - Cores ou marcadores diferenciando os exemplos com diferentes classes.

No vídeo, faça uma análise dos gráficos gerados. Em particular, verifique se os exemplos com baixa confiança e baixa variabilidade coincidem com exemplos rotulados incorretamente ou inconsistentes.

4 Conjuntos desbalanceados

Nesta parte do trabalho, você deve usar o arquivo [A652.pickle](#). Este arquivo contém conjuntos de treino, validação e testes da fonte de dados correspondente². O trecho de código abaixo ilustra como é possível ter acesso aos conjuntos de dados. Nesse trecho de código, `filename` é o nome do arquivo fornecido.

```
import numpy as np
import pickle

f = open(filename, 'rb')

(X_train, y_train, X_val, y_val, X_test, y_test) = pickle.load(f)

print(f"Shapes: ", X_train.shape, X_val.shape, X_test.shape)
```

4.1 Classificação (2 pts)

O atributo alvo corresponde a valores de precipitação (chuva), cuja unidade de medida é mm/h. Você vai notar ao inspecionar as matrizes `y_*` de cada fonte que esses são conjuntos de dados para um problema de regressão (porque o alvo é um valor contínuo). Entretanto, nesta parte do trabalho, você deve alterar o alvo para gerar conjuntos de dados para classificação binária. Para isso, faça o seguinte, para cada arquivo fornecido. Se o alvo for igual a 0, altere para o rótulo **0**; em caso contrário, altere para o rótulo **1**. Após realizar essa transformação, você deve notar que os conjuntos de dados resultantes são extremamente desbalanceados.

Após realizar a transformação descrita acima, você deve investigar se a aplicação de alguma técnica de balanceamento de dados é efetiva no sentido de produzir um modelo de classificação que tenha maior desempenho preditivo. Ou seja, você vai comparar se um modelo treinado sem aplicar balanceamento é pior ou melhor (do ponto de vista preditivo) do que um modelo treinado após a aplicação de alguma técnica de balanceamento. Você deve obrigatoriamente testar as três alternativas de solução descritas em aula (*undersampling*, *oversampling* e *alteração de limiar*), mas está livre para testar outras, se quiser. Faça essa investigação utilizando um único algoritmo de aprendizado, a saber, o `sklearn.ensemble.GradientBoostingClassifier`. Em sua análise dos resultados, forneça as matrizes de confusão obtidas, assim como os relatórios de classificação obtidos por meio da função `classification_report` do Scikit-Learn.

4.2 Regressão (2 pts)

Na Seção 4.1, você enquadrou o problema de prever precipitação como uma tarefa de classificação binária. Nesta parte, você irá implementar um procedimento para criar um modelo de regressão para precipitação. Você deve usar o conjunto de dados fornecido no arquivo `A652.pickle`.

²A fonte desses dados e o modo pelo qual eles foram transformados para geração desses conjuntos são aspectos irrelevantes para o que deve ser feito nesta parte trabalho. Contudo, essas transformações serão alvo de estudo em aulas futuras do curso. Apenas para constar, esses dados correspondem a valores de variáveis meteorológicas (temperatura, pressão atmosférica, humidade relativa do ar, velocidade e direção do vento, precipitação) observados por uma estação de superfície do sistema [INMET](#). O código identificador desta estação é A652.

Você vai notar ao inspecionar as matrizes `y_train`, `y_val`, `y_test`, que elas apresentam valores contínuos não-negativos. Esses são valores de precipitação (chuva) medidos em mm/h. Você vai notar também que a grande maioria dos valores é igual a zero, o que corresponde a uma medição para a variável *precipitação* feita em um instante de tempo em que não está chovendo.

O procedimento para você criar o modelo de regressão é descrito a seguir. Inicialmente você deve criar versões *binárias* das matrizes alvo `y_train`, `y_val`, `y_test`. Concretamente, para cada matriz alvo (`y_train`, `y_val`, `y_test`), substitua todos os valores diferentes de zero por **1** e os valores restantes por **0**. Ao fazer isso, você terá gerado matrizes resposta binárias. Chame essas matrizes de `y_train_bin`, `y_val_bin`, `y_test_bin`, respectivamente. Em seguida, execute os passos listados abaixo.

1. Treine um modelo de classificação binária usando (`X_train`, `y_train_bin`) como conjunto de treinamento. Chame esse modelo de $\mathcal{C}$.
2. Aplique $\mathcal{C}$ a todos os exemplos de `X_train`.
3. Defina `X_train_1` como o subconjunto de `X_train` que foi classificado por $\mathcal{C}$ como sendo da classe **1**. Defina também `y_train_1` como o subconjunto de `y_train` correspondente a `X_train_1`.
4. Treine um modelo de regressão usando (`X_train_1`, `y_train_1`) como conjunto de treinamento. Chame esse modelo de $\mathcal{R}$.
5. Para obter $\mathcal{R}'(\mathbf{x})$, i.e., a previsão de precipitação para um novo exemplo $\mathbf{x}$, inicialmente compute $\mathcal{C}(\mathbf{x})$. Em seguida, use o valor computado para computar $\mathcal{R}'(\mathbf{x})$ da seguinte forma:

$$\mathcal{R}'(\mathbf{x}) = \begin{cases} 0 & \text{se } \mathcal{C}(\mathbf{x}) = 0 \\ \mathcal{R}(\mathbf{x}) & \text{se } \mathcal{C}(\mathbf{x}) = 1 \end{cases}$$

Você está livre para escolher os algoritmos que quiser para construir o classificador $\mathcal{C}$ e o regressor $\mathcal{R}$ mencionados no procedimento descrito acima. Reporte os resultados desse experimento para o conjunto de testes fornecido (`X_test`). Compare seu modelo de regressão produzido por meio desse procedimento com o modelo treinado usando (`X_train`, `y_train`) como conjunto de treinamento. Esse procedimento produziu um melhor modelo do ponto de vista preditivo? Como métrica de avaliação, use RMSE. Reporte valores dessa métrica para cada nível de severidade de precipitação, conforme a Tabela 1.

Tabela 1: Níveis de severidade de eventos de precipitação.

Precipitação (mm/h)	Nível de Severidade
$0 \leq x < 5$	Sem chuva / Leve
$5 \leq x < 25$	Moderada
$25 \leq x < 50$	Forte
$x \geq 50$	Tempestade